Введение в нейронные сети

Лабораторная работа

Тема: Классификация рукописных цифр с использованием многослойного персептрона.

Цель работы: изучить принципы построения и обучения многослойного персептрона для решения задачи классификации изображений. Освоить полный цикл разработки нейросетевой модели – от предобработки данных до оценки её точности и тестирования – в облачной среде *Google Colab*.

Задание: Реализовать модель многослойного персептрона и обучить на готовом датасете *MNIST* для классификации цифр на картинках. Выполнить тестирование обученной нейронной сети. Задание необходимо выполнить в облачной среде разработки *Google Collab*.

Теоретические сведения:

О задаче.

Нужно научить программу по картинке с рукописной цифрой (0-9) определять какая именно цифра изображена. Формально это называется задачей классификации изображений: у нас есть множество объектов, каждый из которых относится ровно к одному из нескольких классов, и мы хотим построить функцию, которая по картинке предсказывает класс.

Для компьютера картинка – не «цифра», а матрица чисел. В нашем случае это изображение 28×28 пикселей – это 784 числа в диапазоне 0-255, где 0 – чёрный пиксель, 255 – белый. Никакого «смысла» в этих числах для машины нет: она видит только набор значений. Задача нейросети – найти в этих числах закономерности, которые устойчиво связаны с правильным ответом.

Почему это вообще нетривиально: одну и ту же цифру разные люди пишут по-разному – разный наклон, толщина штриха, размер, положение на холсте. Модель должна выучить не конкретные картинки из обучающего набора, а общие признаки, по которым цифра «семёрка» отличается от «единицы», – и применить эти признаки к картинкам, которых она раньше не видела. Именно способность работать на новых данных называется обобщением, и это главная цель обучения.

Набор данных для обучения и тестирования нейронной сети (датасет) .

MNIST – классический учебный набор рукописных цифр, собранный из архивов Национального института стандартов и технологий США. Он состоит из 60 000 обучающих и 10 000 тестовых примеров. Каждое изображение – 28×28 пикселей, оттенки серого, цифра по центру. Классы распределены примерно равномерно – каждой цифре соответствует около 6 000 обучающих примеров.

В *Google Colab MNIST* уже доступен – его не нужно скачивать вручную. Он входит в состав библиотеки *Keras* и загружается одной командой

tf.keras.datasets.mnist.load_data(). При первом вызове файл (~11 МБ) скачивается из Интернета и кэшируется; повторные запуски в той же сессии происходят мгновенно. Функция сразу возвращает два готовых набора: обучающий и тестовый, уже разделённые на картинки и метки.

MNIST хорош как учебный датасет по трём причинам: маленький размер (обучение идёт минуты, а не часы), чистые и однотипные данные (нет шума, все картинки уже приведены к одному размеру и центрированы), и при этом задача не тривиальна – даже простые модели показывают точность ~97 %.

Модель нейронной сети.

Начинаем с самой простой архитектуры – *MLP* (*Multilayer Perceptron*, многослойный перцептрон), он же полносвязная сеть. Это сеть из нескольких слоёв, в которой каждый нейрон соединён со всеми нейронами предыдущего слоя.

Идея предельно простая: картинку «разворачиваем» в плоский вектор длиной 784 (просто выстраиваем все пиксели в один ряд), подаём на вход, и сеть через несколько слоёв выдаёт вероятности для каждой из 10 цифр. Ответ – та цифра, вероятность которой оказалась максимальной.

Важно понимать ограничение *MLP*. Для неё порядок пикселей в векторе не важен по существу – важны только значения отдельных чисел. Если перемешать пиксели всех картинок одной и той же случайной перестановкой, *MLP* обучится точно так же хорошо: она не «видит» ни штрихов, ни форм, она запоминает статистику отдельных пикселей. Из-за этого *MLP* – не идеальная модель для картинок, но это обязательная точка отсчёта: любое усложнение (свёрточные сети, *ResNet*) имеет смысл только по сравнению с ней. Если более сложная модель не бьёт *MLP* – значит, усложнение не оправдано.

Любой проект по обучению модели классификации изображений проходит через одни и те же шаги.

Этап 1. Знакомство с данными. Прежде чем что-то обучать, надо посмотреть на данные: сколько примеров, какого размера картинки, в каком диапазоне значения пикселей, равномерно ли распределены классы. Если класс «пятёрка» встречается в 10 раз реже, чем «единица», модель может выучить смещение в сторону частых классов – это надо знать заранее. Полезно и просто глазами посмотреть на несколько картинок: понять, как выглядят данные, которые будет обрабатывать модель.

Этап 2. Предобработка и разделение на выборки. Данные почти всегда нужно подготовить. Минимум два действия: нормализация – приведение значений пикселей к диапазону 0-1 (обучение с большими числами менее стабильно и медленнее), и приведение к нужной форме – для *MLP* картинку надо «развернуть» в вектор длиной 784. Кроме того, данные разделяются на три выборки:

– обучающая (*train*) – на ней модель учится, подбирает веса;

– валидационная (*val*) – на ней проверяем модель в процессе обучения и подбираем гиперпараметры; модель на ней не учится, поэтому оценка честная;

– тестовая (test) – используем для итоговой оценки.

Этап 3. Построение и обучение модели. Собираем архитектуру (для *MLP* – несколько слоёв с функцией активации *ReLU*, на выходе – *softmax* для получения вероятностей) и запускаем обучение. В процессе обучения сеть много раз проходит по обучающей выборке батчами (небольшими порциями), на каждом шаге сравнивает свой ответ с правильным, вычисляет ошибку и чуть-чуть подстраивает веса в сторону уменьшения ошибки.

Этап 4. Оценка и интерпретация. После обучения смотрим на метрики: какова точность на валидации, как менялись *accuracy* (доля правильных ответов) и *loss* (ошибка) по эпохам, нет ли переобучения – ситуации, когда на *train* модель отвечает хорошо, а на *val* – хуже. Графики обучения – главный инструмент, чтобы это увидеть.

Этап 5. Подбор гиперпараметров. Меняем настройки обучения – *learning rate*, число эпох, размер батча, число нейронов – и смотрим, что улучшает результат на валидации. Важно менять по одному параметру за раз, чтобы понимать, что именно помогло.

Этап 6. Финальная проверка на тесте. Один прогон на тестовой выборке – итоговая честная цифра качества модели.

Этап 7. Проверка на «своих» картинках. Рисуем несколько цифр от руки (в *Paint* и загружаем), приводим их к формату *MNIST* и смотрим, как модель с ними справляется. Почти всегда *accuracy* здесь будет ниже, чем на тесте, – и это очень полезный результат: он показывает, что «успех на датасете» и «успех в реальной жизни» – разные вещи.

Как понять, что модель работает хорошо?

Основная метрика для этой задачи – *accuracy* (точность): доля картинок, на которых модель дала правильный ответ. Вторая – *loss* (ошибка): числовая мера того, насколько сильно предсказанные вероятности отличаются от правильного ответа. *Loss* важен, потому что именно его модель напрямую минимизирует, и по его динамике видно, продолжает ли она учиться.

Для *MNIST* ориентиры такие: ниже 95 % – модель явно недоучена; 97-98 % – типичный результат для *MLP*, это наша базовая точка отсчёта; 98,5-99 % – хороший результат для простой свёрточной сети; выше 99 % – уровень современных архитектур. Важно понимать: разница в десятые доли процента между двумя запусками одной и той же модели может быть случайной – из-за инициализации весов и порядка примеров. Чтобы говорить, что модель действительно лучше другой, нужно либо усреднять по нескольким запускам, либо видеть разницу заметно выше уровня шума.

Порядок выполнения задачи

Выполняем в *Google Colab*. Для этого перейдите на сайт: https://colab.research.google.com. Войдите в свой аккаунт (требуется *Google* аккаунт) и создайте новый блокнот (*notebook*).

Для быстрого обучения *MLP* необходимо подключить *GPU*: меню «Среда выполнения → Сменить среду выполнения → аппаратный ускоритель: T4 GPU» («*Runtime → Change runtime type → Hardware accelerator*: *T4 GPU*») → «Сохранить».

Код каждого этапа помещать в отдельную ячейку. Каждую ячейку можно запустить отдельно от остальных (например, при изменении гиперпараметров не нужно выполнять все ячейки, начиная с первой, а только те, которые связаны с обучением и проверкой на тестовой выборке).

Ячейка 1. Импорт библиотек.

Здесь подключаются все инструменты, которые понадобятся дальше: *TensorFlow* и *Keras* для построения и обучения сети, *NumPy* для работы с массивами, *Matplotlib* для графиков и предпросмотра картинок. Дополнительно подавляются информационные сообщения *TensorFlow*, чтобы вывод был чище.

```
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"  # скрыть информационные сообщения TensorFlow

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from keras import layers, models
from google.colab import files  # для загрузки своей картинки в Colab

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

Ячейка 2. Загрузка датасета *MNIST*.

tf.keras.datasets.mnist.load_data() возвращает четыре массива *NumPy*: обучающие изображения (x_*train*), обучающие метки (y_*train*), тестовые изображения (x_*test*) и тестовые метки (y_*test*). В *MNIST* 60 000 обучающих и 10 000 тестовых примеров; каждое изображение – это матрица 28×28 пикселей со значениями от 0 до 255, а метка – целое число от 0 до 9.

```
# Загружаем MNIST: 60 000 обучающих и 10 000 тестовых примеров
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Форма x_train:", x_train.shape)
print("Форма y_train:", y_train.shape)
print("Форма x_test:", x_test.shape)
print("Форма y_test:", y_test.shape)
```

Ячейка 3. Первичный анализ и предпросмотр.

Прежде чем обучать сеть, важно убедиться, что данные корректны: проверить размеры массивов, диапазон значений пикселей и баланс классов (сколько примеров каждой цифры в обучающей выборке). Затем выводится сетка 2×5 с картинками и подписями – чтобы «увидеть» данные глазами. Фиксируется *SEED*, чтобы все последующие случайные операции (например, разделение на валидацию) были воспроизводимы.

```
SEED = 42  # фиксируем случайность для воспроизводимости
np.random.seed(SEED)
tf.random.set_seed(SEED)

# Диапазон значений пикселей до нормализации
print("Мин. и макс. пиксель в x_train:", x_train.min(), x_train.max())

# Баланс классов в обучающей выборке
unique, counts = np.unique(y_train, return_counts=True)
print("Количество примеров каждой цифры в train:")
for digit, count in zip(unique, counts):
    print(f"  {digit}: {count}")

# Сетка 2x5 с картинками и подписями
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(x_train[i], cmap="gray")
    ax.set_title(f"Метка: {y_train[i]}")
    ax.axis("off")
plt.tight_layout()
plt.show()
```

Ячейка 4. Предобработка данных.

Два обязательных шага. Во-первых, значения пикселей приводятся к диапазону 0-1 делением на 255 – это стабилизирует и ускоряет обучение. Во-вторых, изображения «вытягиваются» в плоский вектор длиной 784 (28×28) – именно в таком виде их ожидает полносвязная сеть. Дополнительно от обучающей части отделяется валидационная выборка (10%), которая нужна для честной оценки модели в процессе обучения и подбора гиперпараметров; тестовая выборка до самого конца не трогается.

```
# Нормализация: 0–255 -> 0–1
x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32") / 255.0
```

```
# Вытягивание изображений в плоский вектор длиной 784
x_train_flat = x_train.reshape(-1, 28 * 28)
x_test_flat  = x_test.reshape(-1, 28 * 28)

# Отделяем 10% обучающей выборки в валидационную
val_size = int(0.1 * len(x_train_flat))
x_val = x_train_flat[:val_size]
y_val = y_train[:val_size]
x_train_final = x_train_flat[val_size:]
y_train_final = y_train[val_size:]

print("Размеры выборок:")
print("  train:", x_train_final.shape, y_train_final.shape)
print("  val:  ", x_val.shape, y_val.shape)
print("  test: ", x_test_flat.shape, y_test.shape)
```

Ячейка 5. Настройка гиперпараметров.

Все ключевые параметры вынесены в начало отдельной ячейки, чтобы их было легко менять и сравнивать эксперименты. *EPOCHS* – сколько полных проходов по обучающей выборке сделает сеть. BATCH_*SIZE* – сколько примеров обрабатывается за один шаг обновления весов. LEARNING_*RATE* – размер шага, с которым веса меняются на каждом обновлении. *HIDDEN_UNITS* – число нейронов в скрытом слое.

```
# Гиперпараметры
EPOCHS = 2          # число полных проходов по обучающей выборке
BATCH_SIZE = 128    # число примеров в одном батче
LEARNING_RATE = 0.01  # шаг обучения (чем больше, тем быстрее, но есть риск недообучения)
HIDDEN_UNITS = 128    # число нейронов в скрытом слое
```

Ячейка 6. Построение модели *MLP*.

Создаётся простая полносвязная сеть: входной слой принимает 784 числа (пиксели), один скрытый слой и функцией активации ReLU, выходной слой из 10 нейронов с *softmax* – по одному на каждую цифру. *model*.*summary*() показывает архитектуру и число параметров: это точка отсчёта, с которой позже будут сравниваться свёрточные сети.

```
model = models.Sequential([
    layers.Input(shape=(784,)),          # вход: 784 пикселя
    layers.Dense(HIDDEN_UNITS, activation="relu"),  # скрытый слой
    layers.Dense(10, activation="softmax")          # выход: 10 классов
```

```
])

model.summary()  # покажет слои и число параметров
```

Ячейка 7. Компиляция и обучение.

model.compile() задаёт *optimizer* (*Adam* с выбранным *learning rate*), функцию потерь (*sparse categorical crossentropy* – стандарт для задачи классификации с целочисленными метками) и метрику accuracy. *model.fit*() запускает обучение: на каждой эпохе модель проходит по *train*-выборке батчами, обновляет веса и в конце эпохи проверяется на валидационной выборке. История обучения (*history*) сохраняется – по ней дальше строятся графики.

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=LEARNING_RATE),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    x_train_final, y_train_final,
    validation_data=(x_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)
```

Ячейка 8. Графики обучения.

Два графика – *accuracy* и *loss* – показывают, как модель училась. По ним видно, растёт ли точность на *train* и *val*, не начинает ли *val- loss* расти при продолжающемся падении *train-loss* (это признак переобучения). Эти графики – главный инструмент для интерпретации поведения модели на данном этапе.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# Accuracy
ax1.plot(history.history["accuracy"], label="train")
ax1.plot(history.history["val_accuracy"], label="val")
ax1.set_title("Accuracy по эпохам")
ax1.set_xlabel("Эпоха")
ax1.set_ylabel("Accuracy")
ax1.legend()

# Loss
```

```
ax2.plot(history.history["loss"], label="train")
ax2.plot(history.history["val_loss"], label="val")
ax2.set_title("Loss по эпохам")
ax2.set_xlabel("Эпоха")
ax2.set_ylabel("Loss")
ax2.legend()

plt.tight_layout()
plt.show()
```

Ячейка 9. Оценка на тестовой выборке.

model.evaluate() прогоняет обученную сеть по тестовым данным, которые не участвовали ни в обучении, ни в выборе гиперпараметров. Полученная *accuracy* – это итоговая честная оценка качества *MLP* на *MNIST*.

```
test_loss, test_acc = model.evaluate(x_test_flat, y_test, verbose=0)
print(f"Точность на тестовой выборке: {test_acc:.4f}")
print(f"Loss на тестовой выборке:     {test_loss:.4f}")
```

Ячейка 10. Загрузка своей картинки.

Нарисуйте цифру в *Paint* (чёрным по белому, холст примерно 100×100 пикселей, *PNG*) и загрузите её. Эта ячейка нужна, чтобы затем проверить модель на данных, которые выглядят иначе, чем *MNIST*, и увидеть, как предобработка влияет на результат.

```
# В Colab откроется диалог выбора файла
uploaded = files.upload()

# Получаем имя загруженного файла (предполагаем, что загружен один файл)
image_path = list(uploaded.keys())[0]
print("Загружен файл:", image_path)
```

Ячейка 11. Предобработка своей картинки и предсказание.

Функция *prepare_image*() приводит пользовательскую картинку к формату *MNIST*: переводит в оттенки серого, инвертирует цвета (в *MNIST* цифры белые на чёрном, а в *Paint* – чёрные на белом), изменяет размер до 28×28 и нормализует пиксели к диапазону 0-1. Затем модель выдаёт вероятности для каждой цифры; выводится предсказанная цифра и уверенность модели. Это наглядно показывает, что accuracy на «своих» картинках почти всегда ниже, чем на тестовой выборке.

```
from PIL import Image

def prepare_image(path):
```

```
    """Приводит произвольную картинку к формату MNIST: 28x28, оттенки серого, инверсия, 0-1."""
    img = Image.open(path).convert("L")          # в оттенки серого
    img = img.resize((28, 28))                   # к размеру MNIST
    arr = np.array(img, dtype="float32")
    arr = 255.0 - arr                            # инверсия: в MNIST цифры белые на чёрном
    arr = arr / 255.0                            # нормализация
    return arr.reshape(1, 784)                   # плоский вектор для MLP

# Предобработка и предсказание
x_custom = prepare_image(image_path)
pred_probs = model.predict(x_custom, verbose=0)
pred_digit = np.argmax(pred_probs)
confidence = np.max(pred_probs)

print(f"Предсказанная цифра: {pred_digit}")
print(f"Уверенность модели:  {confidence:.4f}")

# Показываем, как выглядела картинка после обработки
plt.imshow(x_custom.reshape(28, 28), cmap="gray")
plt.title(f"После обработки (предсказание: {pred_digit})")
plt.axis("off")
plt.show()
```